

Smart Driver Monitoring System with Live Alerts Using Flask YOLOv10

Abstract

This project presents a Smart Driver Monitoring System built using Flask and YOLOv10 to detect unsafe driver behaviors through live camera input. The system identifies multiple driver states, including distracted, drowsy, drinking, eating, phone use, safe driving, seatbelt, and smoking. The detection workflow is supported by a custom dataset and a trained YOLOv10 model that processes video frames in a browser interface. When unsafe behavior such as drowsiness or distraction is identified, the system sends a notification email to the administrator for immediate attention. The platform includes user modules such as Home, Register, Login, Detection, and Logout, providing a complete and structured interface for users. The combination of Flask on the backend and HTML, CSS, and JavaScript on the frontend ensures seamless communication between the detection model and the user interface. This system aims to assist in monitoring driver conditions and promoting safer driving practices through automated identification and alerting. Its simple design and efficient processing make it suitable for projects focused on driver behavior analysis, safety systems, and computer-vision-based monitoring frameworks.

Keywords: Driver Monitoring, YOLOv10, Flask, Detection System, Drowsiness, Distraction, Email Alerts, Computer Vision, Safety Monitoring, Behavior Analysis

Introduction

1.1 Problem Statement

Driver behavior errors often lead to unsafe conditions, and identifying such behaviors manually is challenging. Without automated monitoring, unsafe actions such as drowsiness, distraction, phone use, or ignoring safety practices may go unnoticed. Traditional methods lack adaptability and do not provide continuous observation. There is a need for a system that can classify driver actions accurately and respond instantly when an unsafe pattern is detected. The main problem this project addresses is the absence of a unified platform that combines behavior detection, live monitoring, and automated alerts. Additionally, there is a gap in integrating an easy-to-use interface with a strong detection model. Therefore, this project aims to build a system that identifies unsafe driver behaviors through camera-based analysis and sends notification alerts to the administrator to support timely intervention.

1.2 Objective of the project

The primary objective of this project is to build an efficient monitoring system that can classify different driver behaviors using YOLOv10. Another objective is to integrate this model with a Flask framework to achieve smooth interaction between backend processing and the user interface. The project aims to develop a complete workflow in which users can register, log in, and start live detection through a browser. A key objective is to ensure accurate identification of unsafe behaviors, especially drowsiness and distraction, which trigger automatic email notifications to the administrator. Additional goals include presenting detection results clearly, maintaining stable live video processing, and ensuring responsive system behavior. The project also aims to provide a simple, modular structure that can be extended for future work. By combining deep learning, web technology, and automated alerts, the system aims to support research in behavior recognition and safety monitoring.

1.3 Scope of the project

The scope of this project covers the development of an automated driver monitoring system using Flask and YOLOv10. It focuses on detecting a defined set of driver behaviors: distracted, drowsy, drinking, eating, phone use, safe driving, seatbelt, and smoking. The system operates through a live camera feed, making it suitable for continuous monitoring settings. The scope includes creating modules such as Home, Register, Login, Detection, and Logout for managing users and providing an organized interface. The backend handles model integration, frame

processing, and email notifications, while the frontend presents the detection results clearly. The project also involves training or adapting a YOLOv10 model using a labeled dataset from the specified source. The alert mechanism covers unsafe conditions like drowsiness and distraction, ensuring the administrator is informed immediately. Although the system identifies multiple behaviors, its scope is limited to classification and alerting; it does not provide corrective actions or vehicle control. The project is designed for research, experimentation, and demonstration of behavior monitoring using computer vision and deep learning.

1.4 Motivation of the project

The motivation behind developing this system comes from the need to track driver behavior using automated methods. Manual observation or traditional monitoring approaches are often limited, so there is a need for a structured system that can identify unsafe actions without depending on human supervision. Using computer vision techniques and deep learning models enables continuous behavior analysis in a more efficient manner. The intention is to create a system that can recognize specific actions such as drowsiness and distraction and then notify the administrator instantly, helping in early corrective measures. This project also offers a practical way to explore modern detection frameworks, dataset usage, model training, and alert automation. Overall, the motivation is to design a simple, effective, and accessible solution that supports driver safety and contributes to research in automated monitoring technologies.

1.5 Project Introduction

This project introduces a Smart Driver Monitoring System designed to identify various driver behaviors using a live camera feed. It combines deep learning-based detection with a Flask web framework to build a complete and accessible monitoring platform. Drivers exhibit several behaviors that may affect safety, and recognizing these behaviors through automated means helps in timely awareness. The system uses YOLOv10, a high-performance object detection model, trained on a dataset that contains labeled images of different driver actions. These actions include distracted, drowsy, drinking, eating, phone use, safe driving, seatbelt, and smoking.

The system interface is built with HTML, CSS, and JavaScript, providing a smooth and simple user environment. Users begin by registering and logging into the platform, after which they can navigate to the detection module. The backend captures video frames from the camera,

processes them through the YOLOv10 model, and displays the detection results directly in the browser.

A significant feature of this system is its alert mechanism. When the model identifies unsafe behaviors such as drowsiness or distraction, an automatic email is sent to the administrator. This supports timely awareness and helps maintain driver safety monitoring. The inclusion of modules such as Home, Register, Login, Detection, and Logout ensures a complete and structured application flow.

Overall, the project highlights the integration of machine learning, computer vision, and web technologies to build a functional behavior-monitoring system suitable for research and development.

System Analysis

System Analysis

3.1 Existing System

Traditional driver monitoring systems mainly rely on classic computer-vision techniques and simple machine-learning algorithms. These systems often use methods such as Haar cascade classifiers for face and eye detection, HOG (Histogram of Oriented Gradients) for feature extraction, and SVM (Support Vector Machine) classifiers for behavior identification. Some systems track eye closure or head movement using threshold-based rules instead of learning from large datasets. These older approaches work by extracting handcrafted features from the video frames and then applying basic classifiers to determine whether the driver is attentive or not. While such systems can detect limited patterns like closed eyes or head tilting, they struggle to understand complex behaviors such as phone use, drinking, or multitasking. They also rely heavily on fixed lighting, stable face positions, and controlled environments. Because of these limitations, traditional systems cannot achieve high accuracy when dealing with multiple driver behaviors or varying camera conditions.

3.2 Disadvantages of existing system

- Can detect only a few basic actions
- Struggle with poor lighting or unusual face angles
- Cannot handle multiple behaviors at the same time
- Low accuracy due to handcrafted features
- Not suitable for live monitoring with fast movements
- Hard to adapt to different drivers or environments
- Slow processing and limited automation
- No advanced alerting or smart decision-making

3.3 Proposed System

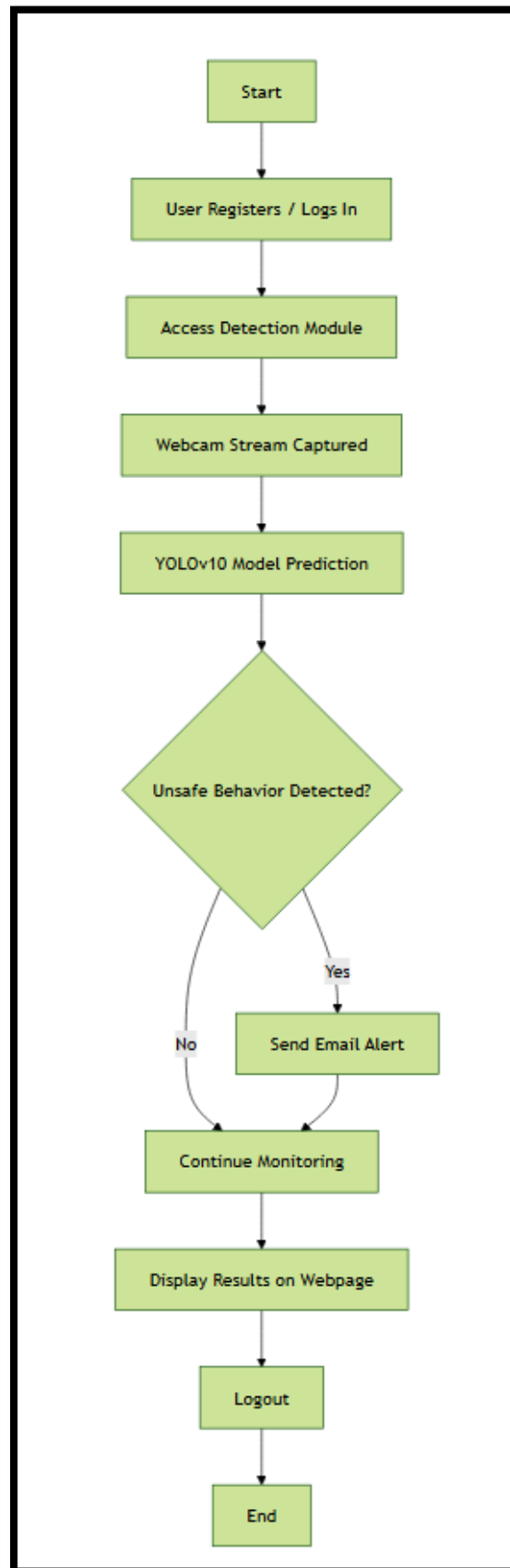
The proposed system introduces an advanced Smart Driver Monitoring System built using Flask and a YOLOv10 detection model. Unlike older methods that depend on handcrafted features, this system uses deep learning to identify multiple driver behaviors directly from live camera input. The YOLOv10 model analyzes each frame in real time and classifies actions such as distracted driving, drowsiness, drinking, eating, phone use, seatbelt status, smoking, and safe driving. The system is integrated into a web interface that includes modules such as Home, Register, Login, Detection, and Logout. After users log in, the live detection page streams video frames to the backend, where the model processes them and returns the prediction results.

A key feature of the system is its alert mechanism. When the model detects unsafe behavior—especially drowsiness or distraction—it automatically sends a notification email to the administrator. This creates a simple but effective safety layer. The system is designed to run smoothly on a web platform, making it easy to operate while providing reliable detection accuracy. The proposed system supports research, improves interaction, and offers a more complete driver behavior analysis setup.

3.4 Advantages:

- Detects many driver behaviors accurately
- Works with live camera feed
- Learns features automatically through deep learning
- More reliable even in changing conditions
- Sends instant alerts for unsafe behavior
- Easy to use through a simple web interface
- Supports secure login and user management
- Flexible design that can be extended in future

3.5 Project flow



SOFTWARE HARDWARE REQUIREMENTS

H/W CONFIGURATION:

Processor	- I3/Intel Processor
Hard Disk	- 160GB
Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- SVGA
RAM	- 8GB

S/W CONFIGURATION:

- Operating System : Windows 7/8/10
- Server side Script : HTML, CSS, Bootstrap & JS
- Programming Language : Python
- Libraries : Flask, Pandas, MySQL. Connector, Scikit-Learn, pytorch
- IDE/Workbench : VS Code
- Technology : Python 3.8+
- Server Deployment : Xampp Server

Implementation

Modules

System Module

1. Data Import

This component handles loading the driver monitoring dataset into the system. The images and labels from the dataset are organized into training, validation, and testing sets. The module ensures proper directory structure and prepares the annotations required by the YOLOv10 model.

2. Preprocessing

Preprocessing involves resizing the images, normalizing pixel values, and applying augmentation techniques such as rotation and brightness adjustment. These steps help maintain consistency across the dataset and improve the model's ability to detect behaviors in different conditions.

3. Model Training

In this phase, YOLOv10 is trained using the preprocessed dataset. Hyperparameters such as batch size, epochs, learning rate, and image dimensions are configured. The model learns to classify various driver behaviors, and the final trained weights are saved for deployment.

4. Prediction Engine

The prediction engine processes incoming frames from the camera during live monitoring. Frames are converted into tensors and passed through the YOLOv10 model. The output includes class labels, bounding boxes, and confidence scores. These results are sent back to the frontend for accurate display.

5. Backend Integration

Flask serves as the backend framework, connecting the model to the web interface. It handles routing, runs the prediction engine, sends detection responses, and manages the email alert system. It also ensures that unsafe behaviors trigger instant notifications to the administrator.

User Module

1. Live Stream Access

Users access the detection page, which activates the webcam feed. The system captures real-time frames and sends them to the backend.

2. Prediction Display

Once predictions are received, the interface displays bounding boxes and labels directly on the video feed. The results update continuously, allowing users to observe detected behaviors instantly.

3. User Interaction

The user interface includes navigation features such as Home, Register, Login, and Logout. Users can interact with buttons, start or stop detection, and view alerts or notifications related to unsafe behavior.

4. Session Management

This module handles user authentication and session control. Only registered users can access the detection page. Flask manages login sessions, maintains secure access, and ensures smooth transitions between modules.

LEARNING OUTCOMES

- Gained understanding of how driver behavior can be monitored using computer vision.
- Learned how to use **YOLOv10** for detecting multiple driver actions in real time.
- Understood how to prepare and preprocess datasets for training deep-learning models.
- Acquired skills in training, testing, and deploying a YOLO-based detection model.
- Learned how to integrate machine-learning models with a **Flask web application**.

- Developed ability to create user modules such as Register, Login, Home, Detection, and Logout.
- Understood how to stream live video from a webcam to a web interface.
- Gained experience in sending **automated email alerts** for unsafe driver behaviors.
- Improved knowledge of frontend development using **HTML, CSS, JavaScript**.